Database Requirement Specifications

Overview of your project idea, describing its functionality & objectives. (Khalid)

Sigil is an academic database system designed to solve two significant challenges in university grading: the hidden manipulation of grades after the official date & the complex transcript sharing system.

In many university databases, administrators can change a student's grade easily, after the official grade set date, with just a simple SQL UPDATE statement, & often, there is no record of that change. This can be quite concerning for students. Moreover, when employers need to verify transcripts, they frequently rely on third-party services. However, students find it difficult to hide less relevant grades without making their official transcripts invalid completely.

Sigil operates as a cryptographic ledger, providing a secure solution. Each grade entry is hashed & organized into a Merkle tree structure, the tree structure many systems like git uses. The university regularly publishes the digitally signed root hash as a Merkle checkpoint. Because every grade is cryptographically linked to that root hash, students can generate a selective disclosure proof, also known as a Merkle proof. This means they can share only specific grades, such as those in computer science when applying for technology jobs, while still proving that the grades they reveal are genuine & unchanged.

The main goals of Sigil are:
1. To ensure that every grade change is evident: grades can only be added, meaning any corrections will leave an approved, auditable trail.
2. To allow outside verifiers to confirm grades using the university's signed root hash without needing to contact the university directly.
3. To give students the ability to disclose a part of their grades while keeping the rest confidential.
4. To maintain a complete history of every grade, so that past versions can be reviewed if necessary.

This system could really transform how grading integrity is maintained & how students present their academic achievements.

Key Entities (minimum 8): Identify the main entities of your system (e.g., Users, Orders, Products, etc.) & their attributes (e.g., name, email, product ID, etc.). (Pratyush)

	Entity	Attributes

1	University	university_id (PK), name, public_key
2	Student	student_id (PK), university_id (FK), full_name (*composite*: first_name, last_name), email, phone_number (*multi-valued*), date_of_birth, gpa (*derived*)
3	Instructor	instructor_id (PK), university_id (FK), full_name (*composite*: first_name, last_name), email, department
4	Course	course_id (PK), university_id (FK), instructor_id (FK), course_code, title, credit_hours, term
5	Grade	grade_id (PK), student_id (FK), course_id (FK), checkpoint_id (FK), supersedes_grade_id (FK to Grade), letter_grade, salt, leaf_hash, leaf_index, change_reason, recorded_at
6	MerkleCheckpoint	checkpoint_id (PK), university_id (FK), term, version_no, root_hash, leaf_count, signature, published_at
7	Verifier	verifier_id (PK), organization_name, contact_email
8	DisclosureProof	proof_id (PK), student_id (FK), verifier_id (FK), status (active, revoked), created_at, expires_at

Grade is the heart of the system: it links a student to a course, & each grade row becomes one leaf of a Merkle tree. MerkleCheckpoint stores the signed root of that tree, & DisclosureProof records which grades a student shared with which verifier.

Relationships: Describe how these entities interact with each other. For example, a “User” can place “Orders,” or a “Product” belongs to a “Category.” (Mohammad)

	Relationship	Cardinality	Participation
1	University admits Student	1 : N	Student mandatory
2	University employs Instructor	1 : N	Instructor mandatory
3	University offers Course	1 : N	Course mandatory

4	Instructor teaches Course	1 : N	Course mandatory, Instructor optional
5	Student receives a grade in Course (through Grade)	M : N	Grade mandatory on both sides
6	Grade supersedes Grade (recursive)	1 : 0..1	Optional
7	University publishes MerkleCheckpoint	1 : N	Checkpoint mandatory
8	MerkleCheckpoint seals Grade	1 : N	Grade optional until published, Checkpoint mandatory (at least one grade)
9	Student creates DisclosureProof	1 : N	Proof mandatory
10	Verifier receives DisclosureProof	1 : N	Proof mandatory
11	DisclosureProof reveals Grade (through DisclosedGrade)	M : N	Proof mandatory (at least one grade)

Business Rules: Any domain-specific rules that affect the design. For example, "An order must contain at least one product," or "A user can have multiple orders but can only place one order at a time." (Khalid)

The most important business rule is that grades are never changed, after the official date, or removed, unless an authorized personnel member changes them; instead, any corrections create a new entry in the database. Each student, instructor, course, & checkpoint is linked to only one university, & an instructor for a course must be part of the same university as that course.

A student can receive only one original grade for each course, & later entries for the same student & course will only be for corrections. The letter grades must be in this list: A, A-, B+, B, B-, C+, C, C-, D+, D, or F.

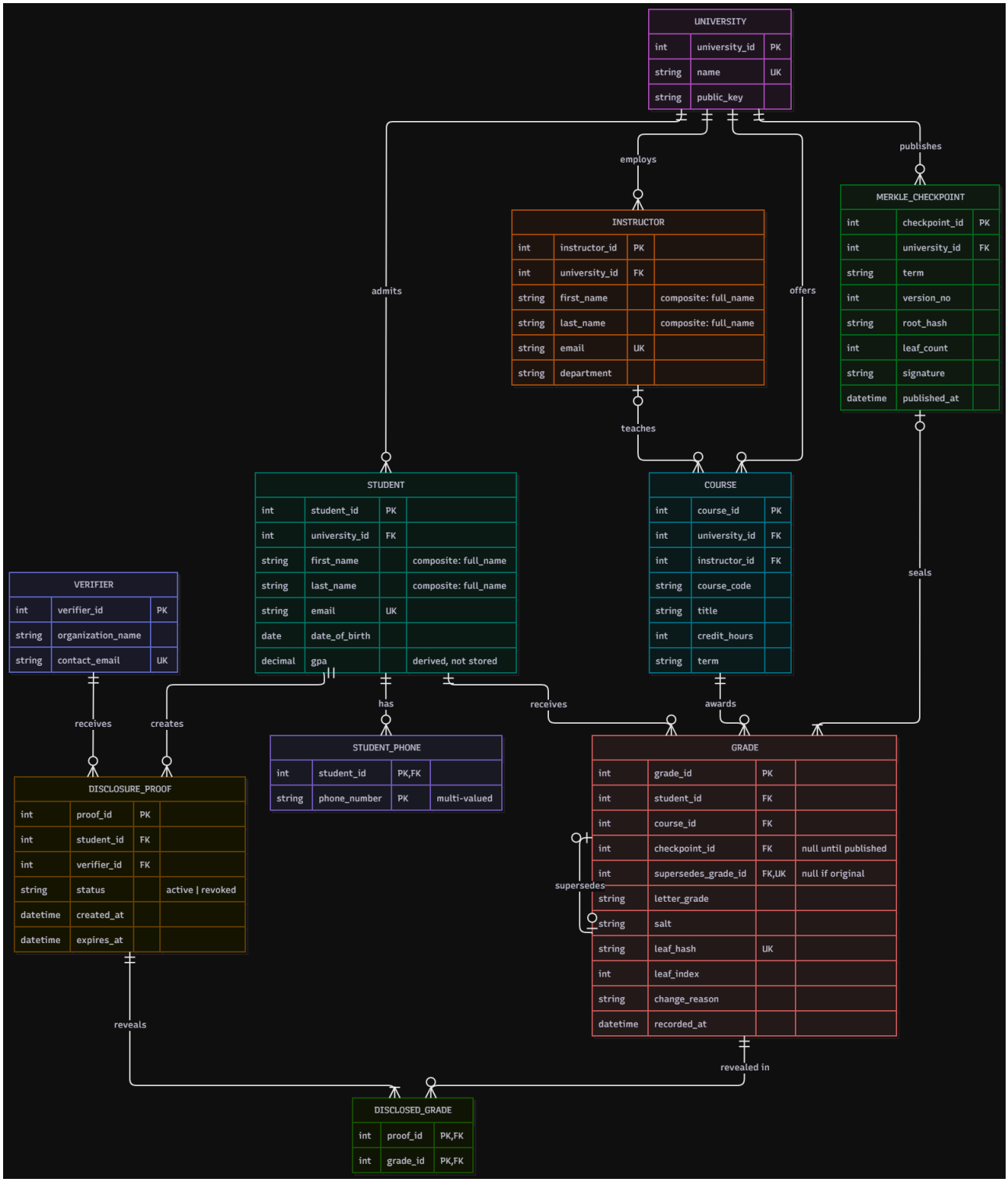
Grade entries can only be added; updates & deletions in the Grade table are not allowed due to database permissions. A correction is a new Grade entry that specifies which previous grade it replaces (supersedes_grade_id) & includes a reason for the change (change_reason). It must relate to the same student & course as the grade it is correcting.

A grade can only be replaced once. The current grade is the one that is not replaced by any other grade. At the end of each term, the university publishes a MerkleCheckpoint, which includes all unpublished grades for that term. Any corrections made afterward will appear in the next checkpoint version for the same term.

The root hash of each checkpoint is signed using the university's private key, which is kept secure & never stored in the database. Each leaf hash contains a random salt for the grade, which means that hidden grades cannot be guessed from a proof.

A disclosure proof must show at least one grade, & every grade revealed in this proof must belong to the student, have been published in a checkpoint, & be the current grade. A proof will no longer be valid if it gets revoked or if the expiration date (expires_at) has passed.

ERD (Harsh): Map all entities & their relationships. Indicate the cardinality (one-to-many, many-to-many) & mandatory relationships. Identify attributes, including PK, FK, & any composite or multi-valued attributes.



Crow's foot notation. || = exactly one (mandatory), o| = zero or one, |{ = one or more (mandatory), o{ = zero or more. PK, FK and UK mark primary, foreign and unique keys. The multi-valued phone_number is shown as its own table (STUDENT_PHONE). DISCLOSED_GRADE resolves the M:N between DISCLOSURE_PROOF and GRADE.

Schema Statements (Pratyush): Convert ERD to RM (Schema Statements): Translate the ERD into relational tables. Define attributes, primary keys, foreign keys, & apply normalization to eliminate redundancy & ensure data integrity. Primary Keys are marked in bold. Foreign Keys are preceded by an '@' symbol

University(**university_id**, name, public_key)
Student(**student_id**, @university_id, first_name, last_name, email, date_of_birth)
Student_Phone(**@student_id, phone_number**)
Instructor(**instructor_id**, @university_id, first_name, last_name, email, department)
Course(**course_id**, @university_id, @instructor_id, course_code, title, credit_hours, term)
MerkleCheckpoint(**checkpoint_id**, @university_id, term, version_no, root_hash, leaf_count, signature, published_at)
Grade(**grade_id**, @student_id, @course_id, @checkpoint_id, @supersedes_grade_id, letter_grade, salt, leaf_hash, leaf_index, change_reason, recorded_at)
Verifier(**verifier_id**, organization_name, contact_email)
DisclosureProof(**proof_id**, @student_id, @verifier_id, status, created_at, expires_at)
DisclosedGrade(**@proof_id, @grade_id**)

Assumptions & Justifications (Mohammad): Include any assumptions made during the design process & justify your decisions (e.g., why you defined certain attributes as composite or multi-valued)

1	Grade acts as the enrollment between Student & Course, & grades are append-only.	An UPDATE would change a leaf already committed to a signed root & lose the old value. Adding a new row that supersedes the old one keeps the full history.

2	Internal Merkle tree nodes are not stored, they are recomputed from the leaf hashes in leaf_index order when a proof is built.	A term has at most a few thousand grades, so rebuilding the tree is fast, & it avoids an extra table.
3	Verifiers do not log in, a student shares a proof with them directly.	Employers should be able to check a transcript without registering.